When a search request is received from a user or a client, the engine forwards that request to one of the available nodes in the cluster. By default, every node in the cluster is assigned to a *coordinator* role, so every node is eligible to pick up client requests on a round-robin basis. Once the request reaches the coordinator node, it determines the nodes on which shards of the applicable documents exist.

In figure 8.1, Node A is the *coordinator* node that receives the client's request. It was chosen as a coordinator node for no reason other than demonstration purposes. Once chosen for the (coordinator) active role, it selects a *replication group* with a set of shards and replicas on individual nodes in a cluster that consists of data. Remember, an index is made of shards, and each shard can exist independently on other nodes. In our example, the index is made of four shards (1, 2, 3, and 4) that exist on Nodes A to D, respectively.

Node A formulates the query request to send to other nodes, requesting them to carry out the search. Upon receiving the request, the respective node performs the search request on its shard. It then extracts the top set of results and responds to the active coordinator with those results. The active coordinator then merges and sorts the data before sending it to the client as a final result.

If the coordinator has a role as a data node, it also digs into its store to fetch results. Not every node that receives the request is necessarily a data node. Similarly, not every node is expected to be part of the replication group for this search query. Now, let's take a moment to load some movie data into our Elasticsearch engine.

8.3 Movie sample data

Let's create some movie test data and movie mappings for this chapter. Because we don't want Elasticsearch to deduce the field types, we provide the relevant data types for each field as mappings when we create the index (especially the `release_date` and `duration` fields, which can't be `text` fields). The following listing shows the `movies` index mapping (if this index exists already, delete it for this chapter's examples by issuing `DELETE movies`).

Listing 8.1 Mappings schema for the `movies` domain

```
PUT movies                          <-- Movies index
{                                   <-- Mappings schema
  "mappings": {
    "properties": {                 <-- Fields and their types
      "title": {
        "type": "text",
        "fields": {                 <-- Multi-field construct
          "original": {
            "type": "keyword"
          }
        }
      },
      "synopsis": {
```